Southeast University

Report on Principles of Compiler

Title LR(1) based Parser

School of Software School (Department) software engineering Major

Student ID 71122135

Student Name Pengyu Xie

Design Location Nanjing

Table

Chapter 1 Syntax Parser Implementation

1.1 Aim

The purpose of this experiment is to construct a syntax analyzer that identifies sentences belonging to a specific language category from an input character stream and generates the corresponding syntax tree. Through the practical implementation of the experiment, the specific goals include the following:

Firstly, to deepen understanding and application of syntax analysis methods. The syntax analyzer is a critical component of a compiler, responsible for organizing the sequence of tokens generated by the lexical analyzer into structures that conform to grammar rules. This experiment enhances our grasp of syntax analysis principles through hands-on development of a syntax analyzer.

Secondly, to explore and understand the impact of custom grammars on sentence structures. Grammar rules determine how sentences are generated and matched. By defining and modifying grammar rules, the experiment helps us analyze the diversity and complexity of sentence structures in a language, providing deeper insights into language design.

Lastly, to master syntax analysis techniques, including top-down and bottom-up parsing methods. By implementing algorithms such as recursive descent parsing, predictive parsing, and LR parsing, the experiment develops practical problem-solving skills for syntax analysis while providing an understanding of the application scenarios and trade-offs of different methods.

This experiment combines theoretical knowledge with practical application, offering a comprehensive opportunity to understand syntax analysis techniques and laying a solid foundation for subsequent study of other modules in compiler design.

1.2 Content description

1) Design a Context-Free Grammar (CFG) that describes the main sentence patterns in programming languages using the tokens defined in Project 1.
2) Construct an LR(1) parsing table for the CFG.
3) Implement the parser based on the LR(1) parsing table.

1.3 Methods

In this experiment, we exclusively employ the LR(1) parsing method to construct a robust and efficient syntax analyzer. The methodology addresses input handling, analyzer structure, state transition logic, shift and reduce operations, error handling, result output, and performance optimization.

These steps ensure the algorithm can correctly and efficiently perform syntax analysis while handling errors effectively.

Firstly, input handling is a critical starting point. The syntax analyzer processes the input character stream by converting it into a data structure that the algorithm can operate on. This includes validating the input format and handling any potential errors to ensure the legality of the input.

Secondly, the analyzer structure is designed to support the core operations of LR(1) parsing. This involves the creation of key data structures, including a state stack, a symbol stack, and the LR(1) parsing table. These structures play essential roles during the analysis process, enabling the reading of character streams and the lookup of state transition tables.

Next, state transition logic is implemented to guide the algorithm through the parsing process. Based on the current state and the input symbol, the analyzer transitions between states using the ACTION and GOTO tables. This step involves performing shift, reduce, or accept actions as determined by the LR(1) parsing table, ensuring a well-defined logical flow for state transitions.

The shift and reduce operations form the backbone of the bottom-up LR(1) parsing approach. During the shift operation, the input symbol is pushed onto the symbol stack, and the state stack is updated according to the state transition table. In the reduce operation, the matched portion of the symbol stack is replaced with the corresponding non-terminal symbol based on the grammar rules.

Error handling is another vital component of the methodology. The algorithm is equipped with a comprehensive error handling mechanism to capture and report errors in the input stream. These errors may include format violations or unrecognizable syntactic structures. The analyzer provides precise feedback to users, including the error location and type, to facilitate debugging and correction.

For valid inputs that conform to the grammar rules, the analyzer outputs the corresponding analysis results. Specifically, the algorithm outputs the sequence of reductions performed during the analysis, reflecting the steps taken to parse the input string according to the grammar.

Lastly, performance considerations are integrated into the methodology. To handle large-scale inputs efficiently, the algorithm incorporates optimizations such as improving data structure design and minimizing unnecessary operations. These measures ensure that the syntax analyzer performs effectively in both time and space complexity.

By addressing these aspects, the experiment achieves a complete and efficient LR(1) syntax analyzer capable of processing input streams, handling errors, and outputting results in a structured and reliable manner.

1.4 Assumptions

This experiment implements an LR(1) parser to analyze a given LR(1) grammar and outputs the corresponding reduction sequence. LR(1) is a bottom-up parsing method capable of processing languages described by context-free grammar (CFG). Its key feature is the ability to make precise parsing decisions at each step based on the current input symbol and the state of the analysis stack.

In this experiment, we adopt the following CFG for syntax analysis:

1) $S \rightarrow$ if (E) S else S

2) S → while (E) S

3) S → E;

4) E → E + E

5) E → E * E

6) E → i

Here, S is the start symbol of the grammar, representing the beginning of a sentence. The non-terminal set is defined as $V_N = \{S, E\}$, and the terminal set is defined as:

$$V_T = \{if, else, while, (,), +, *, i, ;\}.$$

Through the implementation of the LR(1) parser, we can parse input sentences based on this grammar, generate reduction sequences, and gain a deeper understanding of the principles of LR(1) parsing.

In this grammar, the operators + and $*$ are both left-associative, and $*$ has a higher precedence than +. This rule ensures that during the parsing process, multiplication operations are processed before addition operations, adhering to the standard mathematical precedence rules.

1.5 Related LR(1) descriptions

1.5.1 Building the LR(1) Transition Diagram

The principle of the LR(1) parser is based on the LR parsing algorithm, which is a bottom-up parsing method used to process context-free grammars (CFGs). The LR(1) parser constructs an LR(1) canonical collection of sets of items and a corresponding parsing table to determine whether an input string conforms to the given grammar rules. By following this process, the parser can output a reduction sequence or report errors when the input does not match the grammar.

One of the most crucial steps in building an LR(1) parser is the construction of the LR(1) transition diagram. The transition diagram serves as the core data structure of the LR(1) parser, describing all possible state transitions during parsing. Each state in the diagram is represented by a production rule and a dot (•), which indicates the boundary between the recognized portion of the input and the portion yet to be processed. In essence, the transition diagram represents all possible situations the parser might encounter at different stages of analysis.

By constructing the transition diagram, the parser explicitly determines what actions (such as shift, reduce, accept, or error) should be performed for every possible combination of input and state. In this experiment, based on the provided grammar assumptions, the LR(1) transition diagram can be constructed as shown below. This diagram illustrates all item sets in the grammar and their corresponding transitions.

The complete LR(1) transition diagram is shown in Figure 1.1. This diagram illustrates all the item set states of the grammar and their transition relationships. Each state consists of grammar items

and lookahead symbols, with arrows indicating the transition operations between states, such as shift, reduce, and accept. The diagram provides a clear visualization of all possible steps involved in the parsing process, serving as a valuable reference for implementing the LR(1) parser.The diagram consists of 40 states, 2 non-terminal symbols, 9 terminal symbols, and 6 reduction rules, comprehensively illustrating the state transition process of the LR(1) parser.

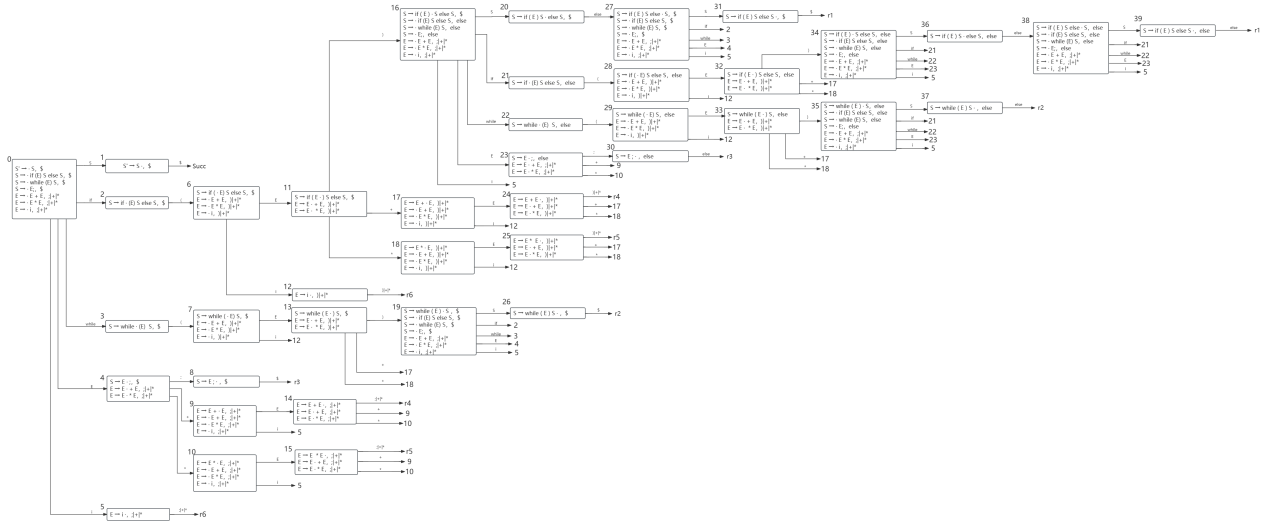

Figure 1.1 Complete LR(1) Transition Diagram

1.5.2 Building the LR(1) Parsing Table

The LR(1) parsing table consists of two parts: the ACTION table and the GOTO table. The ACTION table specifies shift, reduce, or accept actions based on the current state and the next input symbol, while the GOTO table represents state transitions triggered by non-terminal symbols. The construction of these tables is derived from the state transitions in the LR(1) transition diagram shown in figure 1.1. Below table 1.1 、table 1.2 and table 1.3 is the LR(1) parsing table constructed from the diagram.

	Action										Goto	
State	**if**	**else**	**while**	**(**	**)**	**+**	*****	**i**	**;**	**$**	**S**	**E**
0	S2	na	S3	na	na	na	na	S5	na	na	1	4
1	na	na	na	na	na	na	na	na	na	Succ	-	-
2	na	na	na	S6	na	na	na	na	na	na	-	-
3	na	na	na	S7	na	na	na	na	na	na	-	-
4	na	na	na	na	na	S9	S10	na	S8	na	-	-
5	na	na	na	na	na	r6	r6	na	r6	na	-	-

Table 1.1 LR(1) Parsing Table（Part1）

	Action										Goto	
State	**if**	**else**	**while**	**(**	**)**	**+**	*****	**i**	**;**	**$**	**S**	**E**
6	na	na	na	na	na	na	na	S12	na	na	-	11
7	na	na	na	na	na	na	na	S12	na	na	-	13
8	na	na	na	na	na	na	na	na	na	r3	-	-
9	na	na	na	na	na	na	na	S5	na	na	-	14
10	na	na	na	na	na	na	na	S5	na	na	-	15
11	na	na	na	na	S16	S17	S18	na	na	na	-	-
12	na	na	na	na	r6	r6	r6	na	na	na	-	-
13	na	na	na	na	S19	S17	S18	na	na	na	-	-
14	na	na	na	na	na	r4	S10	na	r4	na	-	-
15	na	na	na	na	na	r5	r5	na	r5	na	-	-
16	S21	na	S22	na	na	na	na	S5	na	na	20	23
17	na	na	na	na	na	na	na	S12	na	na	-	24
18	na	na	na	na	na	na	na	S12	na	na	-	25
19	S2	na	S3	na	na	na	na	S5	na	na	26	4
20	na	na	na	na	na	na	na	na	na	na	-	-
21	na	na	na	S28	na	na	na	na	na	na	-	-
22	na	na	na	S29	na	na	na	na	na	na	-	-
23	na	na	na	na	na	S9	S10	na	S30	na	-	-
24	na	na	na	na	r4	r4	S18	na	na	na	-	-
25	na	na	na	na	r5	r5	r5	na	na	na	-	-
26	na	na	na	na	na	na	na	na	na	r2	-	-
27	S2	na	S3	na	na	na	na	S5	na	na	31	4
28	na	na	na	na	na	na	na	S12	na	na	-	32
29	na	na	na	na	na	na	na	S12	na	na	-	33
30	na	r3	na	na	na	na	na	na	na	na	-	-
31	na	na	na	na	na	na	na	na	na	r1	-	-
32	na	na	na	na	S34	S17	S18	na	na	na	-	-
33	na	na	na	na	S35	S17	S18	na	na	na	-	-
34	S21	na	S22	na	na	na	na	S5	na	na	36	23
35	S21	na	S22	na	na	na	na	S5	na	na	37	23

Table 1.2 LR(1) Parsing Table（Part2）

	Action										Goto	
State	**if**	**else**	**while**	**(**	**)**	**+**	*****	**i**	**;**	**$**	**S**	**E**
36	na	S38	na	na	na	na	na	na	na	na	-	-
37	na	r2	na	na	na	na	na	na	na	na	-	-
38	S21	na	S22	na	na	na	na	S5	na	na	39	23
39	na	r1	na	na	na	na	na	na	na	na	-	-

Table 1.3 LR(1) Parsing Table（Part3）

Based on the three tables above, the LR(1) parsing table is constructed. Using this table, an LR(1) parser can be built to read input symbols and perform state transitions based on the information in the **ACTION** and **GOTO**.

1.6 Description of important Data Structures

1.6.1 Storage of CFG Grammar Information

The CFG grammar rules and their corresponding details are stored in a structured format to facilitate efficient parsing and result generation. A one-dimensional string array is used to store the list of grammar rules, while a tuple data structure (`std::tuple`) is employed to represent individual grammar rules. Each tuple contains four elements: the rule number, the reduction result (left-hand side of the production), the string being reduced (right-hand side of the production), and the number of symbols on the right-hand side of the production being reduced. This organization simplifies the management and output of grammar rules during the syntax analysis process. For example, the tuple provides a convenient way to map a reduction step back to its original rule in the grammar.

Figure 1.2 illustrates the data structure used for storing the CFG grammar information, including the array and tuple-based implementation.

```
//规则编号，产生式左侧，产生式右侧，产生式右侧符号个数
tuple<int, char, string, int> grammar[6] = {
    {1, 'S',"if (E) S else S" ,7},
    {2, 'S',"while (E) S" ,5},
    {3, 'S',"E;", 2},
    {4, 'E',"E + E", 3},
    {5, 'E',"E * E", 3},
    {6, 'E',"i", 1}
};
```

Figure 1.2 Implementation of CFG Grammar Storage

1.6.2 Stacks for Symbols and States

In the LR(1) parser, both the symbol stack and the state stack play essential roles in maintaining the parser's state. These stacks are implemented using the last-in, first-out (LIFO) data structure `std::stack`. Each time a symbol is pushed onto the symbol stack, the corresponding state is determined using the state transition table and then pushed onto the state stack. During parsing, if the stack's top forms a handle (i.e., a sequence of symbols matching the right-hand side of a grammar rule), a reduction is performed.

The reduction process involves removing the handle's symbols and states from the stacks and replacing them with the non-terminal symbol resulting from the reduction. To streamline these operations, a custom structure `StackElement` is defined. Each `StackElement` encapsulates an input character and its corresponding state. When a character is read, the state transition table is consulted to determine the next state, and a tuple consisting of the character and its state is pushed onto the stack. During reductions, the elements corresponding to the handle are popped, and the reduced symbol and its resulting state are pushed back. This mechanism ensures an accurate and efficient parsing process.

Figure 1.3 shows the implementation of the `StackElement` structure and how the stacks for symbols and states are managed.

```
//符号栈和状态栈
struct StackElement {
    char character;
    int state;

    StackElement(char ch, int st) : character(ch), state(st) {}
};
```

Figure 1.3 Implementation of Symbol and State Stacks

1.6.3 LR(1) Parsing Table

The LR(1) parsing table is the backbone of the syntax analyzer. It is divided into two sections: the **ACTION** table and the **GOTO** table. The **ACTION** table determines the parser's action based on the current state and the next input symbol. It specifies whether to perform a shift, a reduction, or an acceptance action. On the other hand, the **GOTO** table handles state transitions, guiding the parser on how to move between states based on the non-terminal symbols during reductions.

In the C++ implementation, these tables are stored using the `std::unordered_map` data structure, which provides efficient lookup capabilities. The **ACTION** table uses the current state and terminal symbols as keys, while the **GOTO** table relies on the current state and non-terminal symbols. This separation and organization ensure that the parsing process is both efficient and scalable, capable of handling complex grammars with minimal overhead.

Figure 1.4 and figure 1.5 illustrates the code implementation for constructing and using the LR(1) parsing table, including the `ACTION` and `GOTO` tables.

```
unordered_map<int, unordered_map<string, string>> action_table = {
    {0,  {{"if", "S2"}, {"else", "na"}, {"while", "S3"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {1,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {2,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "S6"}, {")", "na"}, {"+", "na"}, {"
    {3,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "S7"}, {")", "na"}, {"+", "na"}, {"
    {4,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "S9"}, {"
    {5,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "r6"}, {"
    {6,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {7,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {8,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {9,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {10, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "na"}, {"
    {11, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "S16"}, {"+", "S17"},
    {12, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "r6"}, {"+", "r6"}, {"
    {13, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "S19"}, {"+", "S17"},
    {14, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "r4"}, {"
    {15, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")", "na"}, {"+", "r5"}, {"
    {16, {{"if", "S21"}, {"else", "na"}, {"while", "S22"}, {"(", "na"}, {")", "na"}, {"+", "na"},
```

Figure 1.4 Implementation of action Table

```
unordered_map<int, unordered_map<char, int>> goto_table = {
    {0,  {{'S', 1}, {'E', 4}}},
    {1,  {{'S', -1}, {'E', -1}}},
    {2,  {{'S', -1}, {'E', -1}}},
    {3,  {{'S', -1}, {'E', -1}}},
    {4,  {{'S', -1}, {'E', -1}}},
    {5,  {{'S', -1}, {'E', -1}}},
    {6,  {{'S', -1}, {'E', 11}}},
    {7,  {{'S', -1}, {'E', 13}}},
    {8,  {{'S', -1}, {'E', -1}}},
    {9,  {{'S', -1}, {'E', 14}}},
    {10, {{'S', -1}, {'E', 15}}},
    {11, {{'S', -1}, {'E', -1}}},
    {12, {{'S', -1}, {'E', -1}}},
    {13, {{'S', -1}, {'E', -1}}},
    {14, {{'S', -1}, {'E', -1}}},
    {15, {{'S', -1}, {'E', -1}}},
    {16, {{'S', 20}, {'E', 23}}},
    {17, {{'S', -1}, {'E', 24}}},
    {18, {{'S', -1}, {'E', 25}}},
```

Figure 1.5 Implementation of goto Table

1.7 Description of core Algorithms

The syntax analysis program follows a systematic approach based on the grammar rules of the input text. It performs a series of shift and reduce operations to process states and identify the syntax structure of the input. The main steps are as follows:

1. **File Reading and Character Processing** The process begins by opening the specified file path and reading its content character by character. Whitespace and newline characters are ignored,

while the remaining content is processed as individual characters for further analysis.

2. **LR(1) Parsing Table** The LR(1) parsing table is implemented in the program using `action_table` and `goto_table`. The `action_table` stores information about shift and reduce operations, while the `goto_table` maintains state transition information.

3. **State Stack and Action Execution** A stack, `myStack`, is used to simulate state changes during the analysis process. By iteratively reading characters, looking up tables, and performing stack operations, the syntax analyzer attempts to match the grammar structure of the input text. If an unmatched situation occurs, error messages are generated.

4. **Analysis Process** The core of the syntax analysis program operates within an infinite loop, which runs until an acceptance state is reached or an error is detected. During each iteration, the current state and the next input character determine whether a shift or reduce operation is executed.

5. **Shift and Reduce Operations** When the `ACTION` table dictates a shift operation, the character and the next state are pushed onto the stack, with relevant information logged. For reduce operations, the stack pops the required number of symbols and states corresponding to the production rule. The left-hand side of the production is then pushed back onto the stack, and the `GOTO` table is consulted to determine the new state.

6. **Termination State** If the input is successfully matched and the acceptance state (`Succ`) is reached, the program outputs ”Success” and terminates the analysis. If an invalid action or error is encountered, an error message is displayed, and the analysis halts.

The algorithm flowchart for the LR(1) parser is illustrated in Figure 1.6.

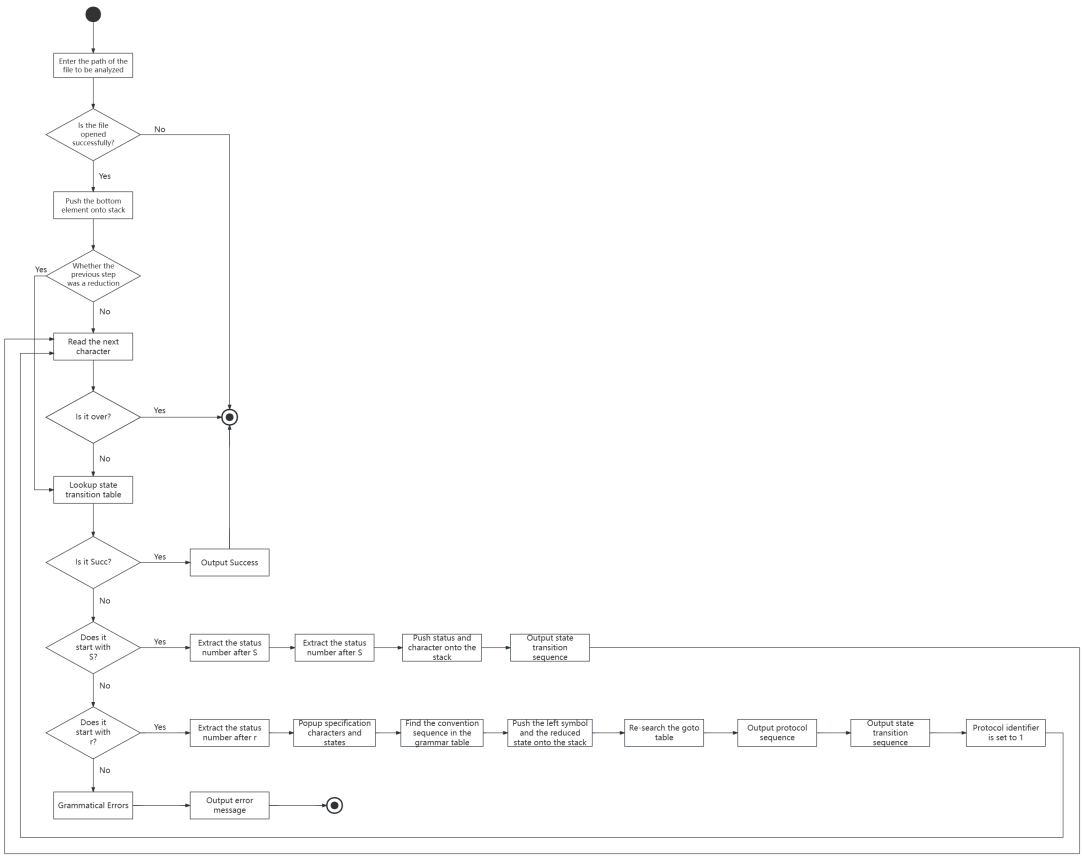

Figure 1.6 Flowchart of the Syntax Analysis Process

1.8 Use cases on running

1.8.1 Correct example

Based on the input test case and the program's output results, it is evident that the execution process of syntax analysis is logically clear and structurally sound. The entire process meticulously records each step, including state transitions, reduction operations, and the final analysis outcome. The output is categorized into the following sections:

1. State Transitions in the ACTION Table (white output): This section displays the type of operation for the current state, such as shift or reduce, and clearly indicates the target state. It effectively demonstrates the practical application of the ACTION table.

2. State Transitions in the GOTO Table (blue output): After completing a reduction, the target state for the corresponding non-terminal symbol is output, showcasing the state transition logic of the GOTO table.

3. Reduction Process (green output): This section illustrates each reduction step, including the matched right-hand side of the production and the resulting non-terminal symbol, providing an intuitive representation of the dynamic execution of the grammar.

4. Successful Analysis Marker (green background output): When the input fully conforms to the grammar rules, the output "Success." is displayed with a green background, signifying that the syntax analysis has been successfully completed.

The output not only meets the experimental objectives but also uses color differentiation to make the analysis process more intuitive. The test results validate the correctness and efficiency of the syntax analyzer, fully demonstrating the functionality of the ACTION and GOTO tables.

Based on the test sample image 1.7 and the test results 1.8, the details of the syntax analyzer's parsing process can be clearly observed. The test sample 1.7 presents a segment of code containing conditional statements and loop structures, designed to cover common language features, including the "if-else" branching statement and the "while" loop. The test results 1.8 meticulously document the state transitions and reduction process of the syntax analysis, ultimately generating the parse tree and outputting the "Success." marker, indicating that the syntax analysis was completed and the code conforms to the grammar rules.

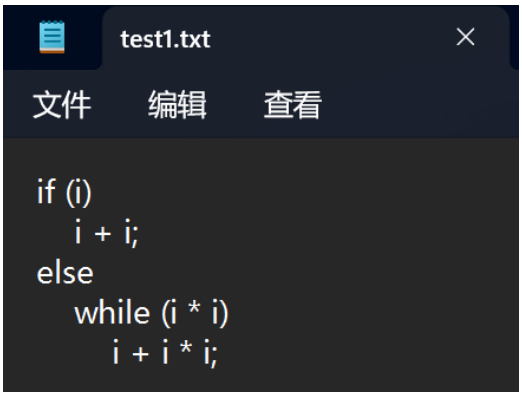

Figure 1.7 Test Sample of LR(1)-Based Syntax Parser

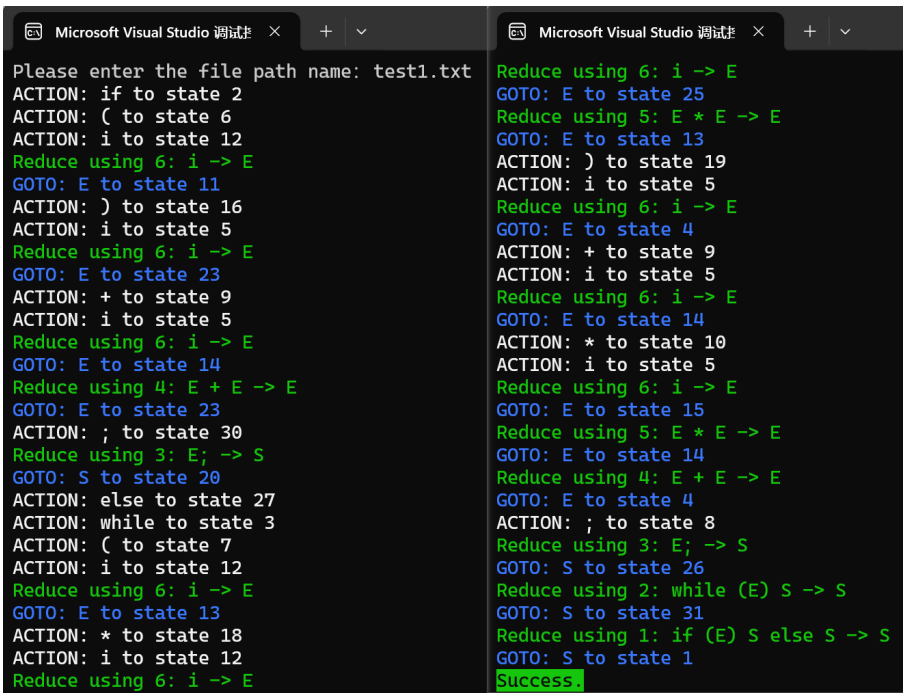

Figure 1.8 Test Result of LR(1)-Based Syntax Parser

1.8.2 Grammatical Errors

The input file, as shown in Figure **??**, contains a simple syntax error: the symbol ‘@‘ following ‘while‘ does not conform to the defined grammar rules. The syntax parser processes the input step by step, recording each action and state transition in the output results, as illustrated in Figure 1.10.

From the output, we can observe that the parser successfully performs multiple shift and reduce operations. For instance, ‘i‘ is reduced to ‘E‘, ‘E + E‘ is reduced to ‘E‘, and keywords like ‘if‘ and ‘else‘ are handled correctly. However, upon encountering ‘while‘, the illegal symbol ‘@‘ leads to an invalid ACTION. The program outputs an error message, ”Invalid action,” along with a helpful hint, ”Maybe near the ’while’ is wrong,” to assist the user in locating the issue.

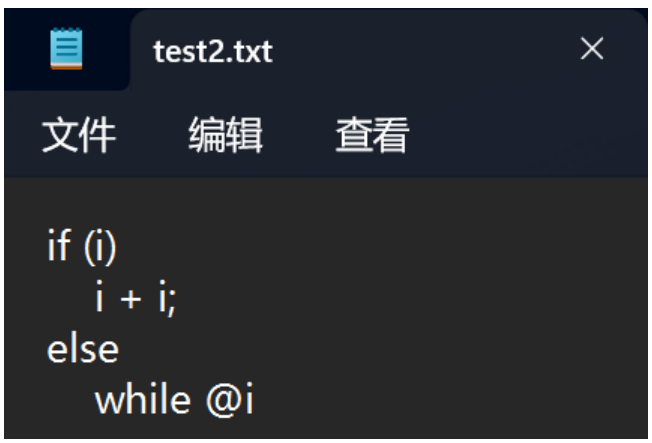

Figure 1.9 the rest about Grammatical Errors

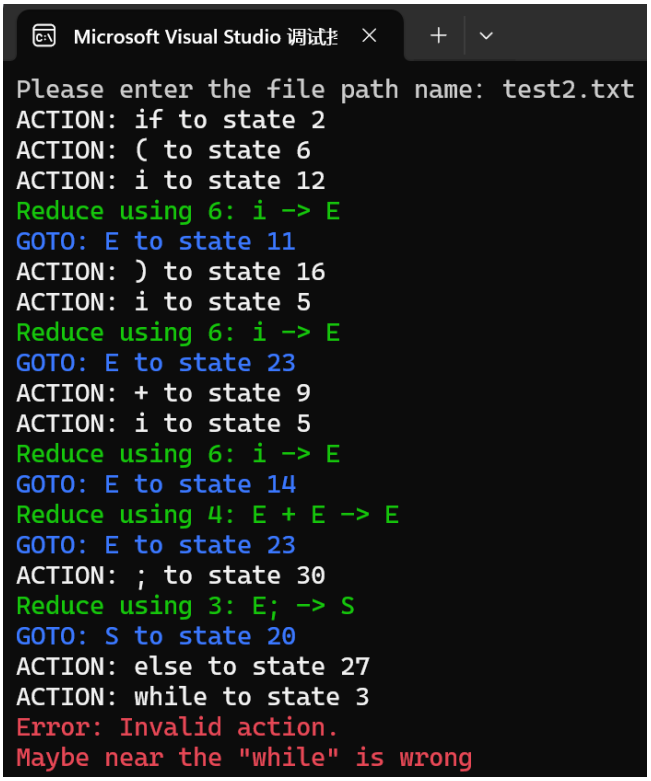

Figure 1.10 the result about Grammatical Errors

1.9 Problems occurred and related solutions

In the design of the syntax parser, I encountered several challenges, primarily related to selecting appropriate data structures and implementing operational logic. Below are the main issues and their corresponding solutions.

1.9.1 Data Structure for the LR(1) Parsing Table

Designing the LR(1) parsing table required an efficient data structure to store the ACTION and GOTO tables. While conceptually a two-dimensional table, the parsing table is implemented using a ternary relationship (current state, input character, next state). To facilitate efficient lookups, I utilized nested mappings: `unordered_map<int, unordered_map<char, string>>` for the ACTION table and `unordered_map<int, unordered_map<char, int>>` for the GOTO table. This structure allows quick and straightforward indexing.

1.9.2 Synchronization of Symbol and State Stacks

The parser maintains both a symbol stack and a state stack, requiring synchronized operations for every shift and reduce step. To simplify implementation, I defined a custom `struct StackElement` that encapsulates both a symbol and a state, thereby unifying the management of stack elements. Although I initially attempted to use a combination of `stack` and `unordered_map`, this approach was complex and error-prone. The introduction of `StackElement` effectively resolved these synchroniza-

tion issues.

1.9.3 Logic for Shift and Reduce Operations

Shift and reduce operations differ significantly in their stack manipulation. A shift operation simply pushes the current character and its corresponding state onto the stack. In contrast, a reduce operation involves:

- Identifying the rule to apply using the ACTION table;
- Popping the right-hand side symbols and their states from the stack;
- Finding the new state using the GOTO table and pushing the reduced symbol and new state onto the stack;
- Keeping the input character unchanged until a shift operation occurs.

To ensure the input character remains unchanged during consecutive reductions, I introduced a boolean flag, `Flag`. When a reduction occurs, `Flag` is set to `false`, skipping character reading in the next iteration. Once the reduction is complete, `Flag` is reset to `true`.

1.9.4 Handling Multi-Character Terminals

An issue arose during testing where keywords such as `if` were incorrectly split into `i` and `f`. To resolve this, I added logic to check for multi-character terminals when encountering potential keywords like `if`, `while`, or `else`. The parser reads additional characters to verify if they match a keyword. If they do, `currentString` is set to the keyword; otherwise, the extra characters are rolled back, and the current character is treated as a single character.

1.9.5 Fixing Errors in Stack Popping During Reduction

During reduction, the stack occasionally popped extra elements due to incorrect counting of right-hand side symbols in the production rule. This issue arose because spaces and multi-character terminals were mistakenly included in the count. To address this, I extended the original tuple (rule number, left-hand side, right-hand side) to a four-element tuple, adding a field to record the precise number of right-hand side symbols. The reduction logic now directly reads this field, ensuring accurate stack operations.

1.10 feelings and comments

During the syntax parser experiment, I gained a profound understanding of the importance of syntax analysis methods and their critical role in compiler design. From the initial stages of the experiment, I began by processing input character streams and defining various sentence structures using context-free grammar (CFG). This step served as the foundation of the experiment, and by carefully

designing grammar rules, I provided the parser with the correct input standards and matching criteria. This experience highlighted to me that the clarity and rationality of grammar rules are essential for the proper functioning of a syntax parser.

Throughout the experiment, I faced numerous challenges. To implement the functionality of the syntax parser, I delved deeply into the theory of LR(1) grammar and the principles behind constructing parsing tables. Based on this understanding, I selected efficient data structures to store the ACTION and GOTO tables, ensuring the accuracy and efficiency of operations within these tables. Managing the complexity of symbol and state stacks, as well as handling the differing effects of shift and reduce operations on the stacks, required me to repeatedly refine my solutions. In particular, the reduction process demanded precise control over stack popping and pushing operations, ensuring the proper matching of states with grammar productions. This experience underscored the importance of both rigorous logic and a strong grasp of data structures in compiler design. Ultimately, through the use of custom structures and logical control, I successfully overcame these challenges, ensuring the correctness and stability of the syntax parser.

Additionally, during the testing phase, I discovered the parser's sensitivity to input characters. For instance, distinguishing single-character terminals from multi-character keywords like 'if' and 'while' required designing additional logic. Addressing this issue not only deepened my understanding of compiler principles but also enhanced my ability to handle details in code design. Furthermore, to resolve errors in the reduction process, I optimized the data structure by upgrading from a three-element tuple to a four-element tuple, thereby ensuring the correctness of the reduction operations. These explorations and practices not only enabled me to grasp the technical essentials of LR(1) syntax analysis but also provided me with a deeper appreciation of how to design robust program logic.

Through this experiment, I not only became more familiar with the theoretical aspects of syntax analysis in compiler principles but also honed my practical skills. I learned how to apply theoretical knowledge to real-world projects and tackle complex issues in compiler design. This experimental experience serves as a valuable guide for my future endeavors in software development and natural language processing. Moreover, it provided me with a more comprehensive understanding of the overall software engineering design process, laying a solid foundation for future learning and research.

Chapter A Source Code for the Syntax Analyzer

Listing A.1 Lexical Analyzer Code

```
#include <iostream>
#include <string>
#include <Windows.h>
#include<fstream>
#include<cstring>
#include <unordered_map>
#include <stack>
#include <tuple>
#include <Windows.h>

using namespace std;

unordered_map<int, unordered_map<string, string>> action_table = {
    {0,  {{"if", "S2"}, {"else", "na"}, {"while", "S3"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
        ↪ "na"}}},
    {1,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "Succ"}}},
    {2,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "S6"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
    {3,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "S7"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
    {4,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "S9"}, {"*", "S10"}, {"i", "na"}, {";", "S8"}, {"$",
        ↪ "na"}}},
    {5,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "r6"}, {"*", "r6"}, {"i", "na"}, {";", "r6"}, {"$",
        ↪ "na"}}},
    {6,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S12"}, {";", "na"}, {"$",
        ↪ "na"}}},
    {7,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S12"}, {";", "na"}, {"$",
        ↪ "na"}}},
```

```
22      {8,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "r3"}}},
23      {9,  {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
           ↪ "na"}}},
24      {10, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
           ↪ "na"}}},
25      {11, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "S16"}, {"+", "S17"}, {"*", "S18"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "na"}}},
26      {12, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "r6"}, {"+", "r6"}, {"*", "r6"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "na"}}},
27      {13, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "S19"}, {"+", "S17"}, {"*", "S18"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "na"}}},
28      {14, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "r4"}, {"*", "S10"}, {"i", "na"}, {";", "r4"}, {"$",
           ↪ "na"}}},
29      {15, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "r5"}, {"*", "r5"}, {"i", "na"}, {";", "r5"}, {"$",
           ↪ "na"}}},
30      {16, {{"if", "S21"}, {"else", "na"}, {"while", "S22"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
           ↪ "na"}}},
31      {17, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S12"}, {";", "na"}, {"$",
           ↪ "na"}}},
32      {18, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S12"}, {";", "na"}, {"$",
           ↪ "na"}}},
33      {19, {{"if", "S2"}, {"else", "na"}, {"while", "S3"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
           ↪ "na"}}},
34      {20, {{"if", "na"}, {"else", "S27"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "na"}}},
35      {21, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "S28"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "na"}}},
36      {22, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "S29"}, {")",
           ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
           ↪ "na"}}},
37      {23, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
           ↪ "na"}, {"+", "S9"}, {"*", "S10"}, {"i", "na"}, {";", "S30"}, {"$",
```

```
        ↪ "na"}}},
38      {24, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "r4"}, {"+", "r4"}, {"*", "S18"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
39      {25, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "r5"}, {"+", "r5"}, {"*", "r5"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
40      {26, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "r2"}}},
41      {27, {{"if", "S2"}, {"else", "na"}, {"while", "S3"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
        ↪ "na"}}},
42      {28, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S12"}, {";", "na"}, {"$",
        ↪ "na"}}},
43      {29, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S12"}, {";", "na"}, {"$",
        ↪ "na"}}},
44      {30, {{"if", "na"}, {"else", "r3"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
45      {31, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "r1"}}},
46      {32, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "S34"}, {"+", "S17"}, {"*", "S18"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
47      {33, {{"if", "na"}, {"else", "na"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "S35"}, {"+", "S17"}, {"*", "S18"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
48      {34, {{"if", "S21"}, {"else", "na"}, {"while", "S22"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
        ↪ "na"}}},
49      {35, {{"if", "S21"}, {"else", "na"}, {"while", "S22"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
        ↪ "na"}}},
50      {36, {{"if", "na"}, {"else", "S38"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
51      {37, {{"if", "na"}, {"else", "r2"}, {"while", "na"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}},
52      {38, {{"if", "S21"}, {"else", "na"}, {"while", "S22"}, {"(", "na"}, {")",
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "S5"}, {";", "na"}, {"$",
        ↪ "na"}}},
53      {39, {{"if", "na"}, {"else", "r1"}, {"while", "na"}, {"(", "na"}, {")",
```

```
        ↪ "na"}, {"+", "na"}, {"*", "na"}, {"i", "na"}, {";", "na"}, {"$",
        ↪ "na"}}}
};

unordered_map<int, unordered_map<char, int>> goto_table = {
    {0, {{'S', 1}, {'E', 4}}},
    {1, {{'S', -1}, {'E', -1}}},
    {2, {{'S', -1}, {'E', -1}}},
    {3, {{'S', -1}, {'E', -1}}},
    {4, {{'S', -1}, {'E', -1}}},
    {5, {{'S', -1}, {'E', -1}}},
    {6, {{'S', -1}, {'E', 11}}},
    {7, {{'S', -1}, {'E', 13}}},
    {8, {{'S', -1}, {'E', -1}}},
    {9, {{'S', -1}, {'E', 14}}},
    {10, {{'S', -1}, {'E', 15}}},
    {11, {{'S', -1}, {'E', -1}}},
    {12, {{'S', -1}, {'E', -1}}},
    {13, {{'S', -1}, {'E', -1}}},
    {14, {{'S', -1}, {'E', -1}}},
    {15, {{'S', -1}, {'E', -1}}},
    {16, {{'S', 20}, {'E', 23}}},
    {17, {{'S', -1}, {'E', 24}}},
    {18, {{'S', -1}, {'E', 25}}},
    {19, {{'S', 26}, {'E', 4}}},
    {20, {{'S', -1}, {'E', -1}}},
    {21, {{'S', -1}, {'E', -1}}},
    {22, {{'S', -1}, {'E', -1}}},
    {23, {{'S', -1}, {'E', -1}}},
    {24, {{'S', -1}, {'E', -1}}},
    {25, {{'S', -1}, {'E', -1}}},
    {26, {{'S', -1}, {'E', -1}}},
    {27, {{'S', 31}, {'E', 4}}},
    {28, {{'S', -1}, {'E', 32}}},
    {29, {{'S', -1}, {'E', 33}}},
    {30, {{'S', -1}, {'E', -1}}},
    {31, {{'S', -1}, {'E', -1}}},
    {32, {{'S', -1}, {'E', -1}}},
    {33, {{'S', -1}, {'E', -1}}},
    {34, {{'S', 36}, {'E', 23}}},
    {35, {{'S', 37}, {'E', 23}}},
    {36, {{'S', -1}, {'E', -1}}},
    {37, {{'S', -1}, {'E', -1}}},
    {38, {{'S', 39}, {'E', 23}}},
    {39, {{'S', -1}, {'E', -1}}}
};

```

```
//规则编号，产生式左侧，产生式右侧，产生式右侧符号个数
tuple<int, char, string, int> grammar[6] = {
    {1, 'S',"if (E) S else S" ,7},
    {2, 'S',"while (E) S" ,5},
    {3, 'S',"E;", 2},
    {4, 'E',"E + E", 3},
    {5, 'E',"E * E", 3},
    {6, 'E',"i", 1}
};

//符号栈和状态栈
struct StackElement {
    char character;
    int state;

    StackElement(char ch, int st) : character(ch), state(st) {}
};
stack<StackElement> myStack;

const int ERROR_STATE = -1;
const int ACCEPT_STATE = -2;

void syntaxAnalysis(const string& filePath) {
    ifstream inFile(filePath);
    if (!inFile.is_open()) {
        cout << "Error: Unable to open file." << endl;
        return;
    }
    char currentChar;
    string currentString;
    int currentState = 0; // 初始状态
    myStack.push(StackElement('$', 0)); // 将初始状态压入状态栈
    inFile.seekg(0, ios::beg);
    string token; // 用于存储识别的单词或词法单元
    bool Flag = true;
    while (1) {
        if (Flag) {
            inFile.get(currentChar);
            if (!inFile.eof()) {

                // 根据当前状态和读入的字符执行相应的动作
                if (currentChar == ' ' || currentChar == '\n') {
                    continue; // 忽略空格和换行符
                }
                else if (currentChar == 'i') {
                    char nextChar;
                    if (inFile.get(nextChar)) {
```

```
                    if (nextChar == 'f') {
                        currentString = "if";
                    }
                    else {
                        inFile.putback(nextChar);
                        currentString = 'i';
                    }
                }
                else {
                    currentString = "i"; // 读取失败, 设置为单字符
                }
            }
            else if (currentChar == 'e') {    // 可能是 else
                char nextChar1, nextChar2, nextChar3;
                if (inFile.get(nextChar1) && inFile.get(nextChar2) &&
                    ↪ inFile.get(nextChar3)) {
                    if (nextChar1 == 'l' && nextChar2 == 's' && nextChar3
                        ↪ == 'e') {
                        currentString = "else"; // 匹配 else
                    }
                    else {
                        // 回退字符
                        inFile.putback(nextChar3);
                        inFile.putback(nextChar2);
                        inFile.putback(nextChar1);
                        currentString = "e"; // 单字符 e
                    }
                }
                else {
                    currentString = "e"; // 读取失败, 设置为单字符
                }
            }
            else if (currentChar == 'w') {   // 可能是 while
                char nextChar1, nextChar2, nextChar3, nextChar4;
                if (inFile.get(nextChar1) && inFile.get(nextChar2) &&
                    ↪ inFile.get(nextChar3) && inFile.get(nextChar4)) {
                    if (nextChar1 == 'h' && nextChar2 == 'i' && nextChar3
                        ↪ == 'l' && nextChar4 == 'e') {
                        currentString = "while"; // 匹配 while
                    }
                    else {
                        // 回退字符
                        inFile.putback(nextChar4);
                        inFile.putback(nextChar3);
                        inFile.putback(nextChar2);
                        inFile.putback(nextChar1);
                        currentString = "w"; // 单字符 w
```

```
                    }
                }
                else {
                    currentString = "w"; // 读取失败，设置为单字符
                }
            }
            else if (currentChar == '(' || currentChar == ')' ||
                ↪ currentChar == '+' || currentChar == '*' || currentChar
                ↪ == ';') {
                currentString = currentChar;
            }
            else if (currentChar == '$') {
                currentChar = '$'; // 设置当前字符为结束符号
                currentString = '$';
            }
        }
        else {
            currentChar = '$';
            currentString = '$';
        }
    }

    // 根据读入字符和当前状态查找LR(1)分析表
    string action;

    // 小写字符，从ACTION表中查找操作
    action = action_table[currentState][currentString];
    Flag = true;
    if (action == "Succ") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
            ↪ BACKGROUND_GREEN | BACKGROUND_INTENSITY);
        cout << "Success.";
        return; // 接受状态，停止分析
    }
    else if (action[0] == 'S') {
        // 移进操作
        int nextState = stoi(action.substr(1)); // 下一个状态
        myStack.push(StackElement(currentChar, nextState)); //
            ↪ 将当前字符和下一个状态压入栈
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
            ↪ FOREGROUND_INTENSITY | FOREGROUND_RED | FOREGROUND_GREEN |
            ↪ FOREGROUND_BLUE);   //字体恢复原来的颜色
        cout << "ACTION: " << currentString << " to state " << nextState <<
            ↪ endl;
        currentState = nextState;

    }
```

```
229         else if (action[0] == 'r') {
230             // 规约操作
231             int ruleIndex = stoi(action.substr(1)); // 规则编号
232             int ruleNum = get<0>(grammar[ruleIndex - 1]); // 规则编号
233             char left = get<1>(grammar[ruleIndex - 1]); // 规约左部符号
234             string right = get<2>(grammar[ruleIndex - 1]); // 规约右部符号
235             int ruleLen = get<3>(grammar[ruleIndex - 1]);//规约右部符号个数
236             // 进行规约操作，弹出规约的字符和状态
237             for (size_t i = 0; i < ruleLen; ++i) {
238                 myStack.pop();
239             }
240             // 获取当前状态并查找规约后的状态
241             currentState = myStack.top().state;
242             char leftSymbol = left;
243             int nextState = goto_table[currentState][leftSymbol];
244             myStack.push(StackElement(leftSymbol, nextState)); //
                ↪ 将左部符号和规约后的状态压入栈
245             SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
                ↪ FOREGROUND_INTENSITY | FOREGROUND_GREEN);  //绿色
246             cout << "Reduce using " << ruleNum << ": " << right << " -> " <<
                ↪ left << endl;
247             SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
                ↪ FOREGROUND_INTENSITY | FOREGROUND_BLUE);   //蓝色
248             cout << "GOTO: " << leftSymbol << " to state " << nextState << endl;
249             currentState = nextState;
250             Flag = false;
251         }
252         else {
253             SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
                ↪ FOREGROUND_INTENSITY | FOREGROUND_RED);    //红色
254             cout << "Error: Invalid action." << endl;
255             cout << "Maybe near the \"" << currentString << "\" is wrong" <<
                ↪ endl;
256             return; // 遇到错误，停止分析
257         }
258
259     }
260
261     inFile.close(); // 关闭文件
262 }
263
264 int main() {
265     string filename;
266     cout << "Please enter the file path name: ";
267     cin >> filename;
268     syntaxAnalysis(filename);
269     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
```

```
        ↪ FOREGROUND_INTENSITY | FOREGROUND_RED | FOREGROUND_GREEN |
        ↪ FOREGROUND_BLUE);    //字体恢复原来的颜色
270     return 0;
271 }
```